Chap 16 - Smart Pointers

1. Do the following
    a. Create a unique_ptr to an integer called *p1* but do not assign it.
    b. Using an *if* statement check if *p1* has been assigned. If it hasn't, then then use make_unique to allocate an integer and initialize it to 42. Print out the value that *p1* points to
    c. Create a unique_ptr to int called *p2* and assign it the value of 99;
    d. Swap *p1* and *p2*
    e. Create a unique_ptr called *p3* to an array of integers with 20 elements. Initialize each element to its index value(using for loop).
    f. Make a function that returns a unique_ptr to int and initializes the value to 42. Call and test the function.

2. Make the following class:

```
struct A{
    int x;
    virtual void callme() { cout << "A\n"; }
    virtual ~A() { cout << "Agone\n"; }
};

struct B : public A {
    virtual void callme() { cout << "B\n"; }
    ~B() { cout << "Bgone\n"; }
};
```

    a. Create a unique_ptr<A> called *p* but assign it to a unique_ptr<B> using make_unique. Does this work and why
    b. Call the *callme()* method of *p*. What does it print?
    c. Set *p*'s '*x*' member to 42
    d. Assign *p* to equal nullptr. What happens to the old allocated memory *p* had pointed to?
    e. Assign *p* to a unique_ptr<B> using make_unique again. Then assign *p* to equal a new pointer using make_unique. What happens to the old allocated memory *p* had pointed to?
    f. Create a new pointer called *parr* that points to an array of 10 elements of B objects. Assign each object's x value to 42;
    g. As you saw setting a pointer to nullptr will free up the allocated memory. Also the object/memory is freed when the pointer goes out of scope. What is another way to free up what the pointer points to? Use it to free up *parr*… what does it print?

3. Create your own String class (you can use the one you made a while ago in homework). Instead of using an actual pointer to an array of char make a unique_ptr to an array of char. Make sure it has assignment operators and copy constructors. One thing to add is a method called data() that returns a const pointer (a real pointer not a smart pointer) to the array of char that represents the string.

4. Create a set of unique_ptr<A> but assign unique_ptr's to B. What must you implement to make the set work?

5. Create a **set** of the unique pointers to class A objects you made in the previous problem and call it '*set_unique1'*.
    a. Create a for loop from 0 to 9 that inserts unique pointers into *set_unique1* with the value of the for loop variable (so you should have a set of unique pointers with the x values set to (0…9)).
    b. Create another set of unique pointers to integers and call it *'set_unique2'.*
    c. Let's try to copy over the unique pointer from *set_unique1* to *set_unique2* using the example:
    set_unique2.insert(set_unique1.begin(), set_unique1.end());

    What happens? Does it compile? Why does it make sense that it fails??
    d. Now change your compiler to compile C++17
    e. Comment out the insert line from problem c). Instead use the new C++17 *set* method called *merge().* Use merge to move the values from *set_unique1* to *set_unique2.*
    What happens to both *set_unique1* and *set_unique2*??

    Before C++17 you would have to manually move each element of the set over to the other set (something we will see soon with the std:move() function).

6. It is a common practice to create a 'factory' function (or class). The job of the factory function is to create and return an object based on a set of arguments given to the function.

    For example say you have a Shape class and three subclasses derived from Shape: Quadrilateral, Ellipse and Triangle.

    Make a factory function that takes as an argument a string and based on the string value (could ne "Quadrilateral", "Ellipse", or "Triangle") return a unique pointer to a Shape that has been constructed of the type passed.

    Example:

```
unique_ptr<Shape> p = MakeShape("Triangle");
// p will be a unique pointer to a Triangle class.
```

7. Make a unique pointer to the lambda function: [](int x){return 42 + x}
    Call the lambda function through the unique pointer with an argument of 10 and print the result.

Shared Pointers

8. Do the following
    a. Create a shared_ptr called '*test*' to an int with an initial value of 0.
    b. Create a shared_ptr called '*pa*' to class A from problem 2.
    c. Set *pa*'s x member variable to 42 and then call the *callme()* method.
    d. Create another shared_ptr to class A called '*pa2*' and assign it to '*pa*'.
    e. Print out the reference count of '*pa*'. Then set *pa2* to equal nullptr and print out the reference count to *pa*. What happens?
    f. Set *pa* to nullptr. What does that trigger?
    g. Set *pa* to a new shared pointer and then call pa's reset method.
    h. If you needed to get the physical pointer to the memory *pa* pointed to what method do you call?

9. Say I have the following line of code:

```
unique_ptr<string> Ted = make_unique<string>(string("Ted"));
```

Let's say I want the string that Ted points to to now be a shared pointer. How would I convert this efficiently (there are a number of ways such as calling std::move() but that is in a later chapter… you can do this with the methods of the unique pointer class.)

10. A smart pointer(shared or unique) to a lambda function is possible.. under certain circumstances. At first one might think the following will work (in this case the ***using*** keyword is used in a similar fashion as ***typedef***):

```
using lambtype = decltype([](int x, int y) {return 42; });
auto shlamb = make_shared<lambtype>([](int x, int y) { return x + y; });
auto unlamb = make_unique<lambtype>([](int x, int y) { return x - y; });
```

Unfortunately it fails miserably, and the reason is not terribly obvious. A *decltype()* on a lambda function always returns a different type for different lambdas… they simply won't be the same even though they have the same function arguments and return type.

In this case (and only in the case where the capture clause is empty), we can convert a lambda function to a function wrapper class. A function wrapper class (#include <functional>) wraps any callable object (function, lambda, functor) into an object so long as it matches the definition.

We can make a slight change to the above code:

```
using lambtype = function<int(int,int)>;
auto shlamb = make_shared<lambtype>([](int x, int y) { return x + y; });
auto unlamb = make_unique<lambtype>([](int x, int y) { return x - y; });
```

---->Your exercise is to determine HOW to call the lambda functions pointed to by *schlamb* and *unlamb*.

Also note… having smart pointers to lambda functions may not be terribly useful because there is no memory allocation to worry about so 'why bother?' What is useful would be to have a vector of lambda functions and the code above is close to accomplishing that.

Weak Pointers

11. Do the following:
    a. Create a shared_ptr to an integer named '*sp*'. Set its value to 42
    b. Create a weak_ptr to integers named '*wp*' that points to *sp*.
    c. Print out the weak pointer's use_count and the value *wp* points to
    d. Create a new shared pointer named *sp2* from *wp*.
    e. Set *sp* to nullptr or call its reset() method. Check if *wp* is expired.
    f. Set *sp2* to nullptr or call its reset() method. Check if *wp* is expired.

12. Why is the following code a very very VERY bad idea? What change could you make to fix it?

```
struct circ {
public:
    shared_ptr<circ> sptr;
    void setp(shared_ptr<circ>& p) { sptr = p; }
    ~circ() { cout << "circ deleted\n"; }
};

auto cp = make_shared<circ>();
cp->setp(cp);
cp.reset();
```

13. Make a class called my_list that is a singly linked list of class A objects from problem 2 using shared_ptrs's. In your class have a shared_ptr called "*head*" that points to the head first node in the list. You will need to create a "*node*" class that contains the A object and the shared_ptr to the next elements in the list.

Create two methods
    a. AddFront() - adds a new node to the front of the list
    b. Clear() – erases the whole list by simply setting the head shared_ptr to nullptr. Test that this is true (remember an A object prints out "Agone" in its destructor).
    c. Modify my_list to be a doubly linked list. Do NOT modify the Clear() method… it should still delete all items by simply setting the head member to nullptr. Obviously you will need to use weak_ptrs to accomplish this.

C++ has very bad acronyms: C++ has possibly the worst acronyms of any programming language. I am going to introduce you to two very important but terribly named acronyms: *Pimpl* and *RAII*.

Pimpl stands for 'pointer to implementation' and is a programming technique that places the implementation of a class(the interface class) into another class(the 'implementation' class). The user calls methods in the interface class that in turns call methods in the implementation class. The 'interface' class accesses the 'implementation' class via a pointer (hence the name pointer to implementation).

Why do this?? When you start writing much larger professional code and you have multiple libraries you start to hit issues with the ABI (Application Binary Interface). Say for example I have a class in a library that is used in many applications. If I add a new method or member that will affect the size and layout in memory of the object. This will 'break' all the dependent code(the code that calls the library) and I'll have to recompile **everything** that uses that library.

This also can reduce compiler build times (something very very important on big projects) because the implementation class doesn't need to be in the header file… it can stay local to the interface class so if any changes are made only that one file will need to be compile.

The thing is the implementation class needs to also contain the data members as well and so must be deleted properly. This is where unique pointers come into play… if I have a pointer to an implementation class I need to destroy it properly when the interface class is deleted. Using a unique pointer basically does this automatically (or automagically).

Here is small example of the header file:

```
# pragma once
// Header.h
# include <memory>
# include <string>
using namespace std;

class employee {
private:
    struct impl;                    // this class is solely in the cpp file - makes compiling faster
    unique_ptr<impl> pimpl;         // pointer to implementation
public:
    employee(string name,int id);
    employee(const employee& other);
    employee& operator=(employee emp);
    ~employee()=default;
    int getid();
    void setid(int);
};
```

And here is an example of the cpp file:

```
# include <iostream>
# include "Header.h"

struct employee::impl {
    impl(const string& name, int id)
        : name(name),id(id) {}
    ~impl() = default;
    string name;
    int id;
};

// Constructor connected with our impl structure
employee::employee(string name, int id)
    : pimpl(make_unique<impl>(name,id)) {}

employee::employee(const employee& other)
    : pimpl(new impl(*other.pimpl)) {}

employee& employee::operator=(employee emp) {
    swap(pimpl, emp.pimpl);
    return *this;
}

int employee::getid() {
    return pimpl->id;
}

void employee::setid(int id) {
    pimpl->id = id;
}

int main() {
    employee bob("Bob",10), joe("Joe",0);
    employee temp(bob);

    joe.setid(42);
    cout << joe.getid();
    bob = joe;

}
```

14. Modify the employee implementation and interface classes by:
    a. Add in methods for setting and getting the employee tier level (integer type). A tier level would go from 1 to 10 (10 is the CEO… 1, would be 'junior programmer' (haha)).
    b. Add in a methods for setting and getting the number of years worked(an integer type).
    c. Make a method called calc_salary() in the ***implementation class*** that calculates the salary using the employee level and the number of years worked. Use the following formula: salary = 50,000 + (10,000 * tier level) + (2000 * number of years worked).

d. Make a method called calc_salary in the ***interface class*** that simply returns the value from the calc_salary in the implementation class.

T**he second bad acronym is RAII (*Resource Acquisition Is Initialization*).** This is a great C++ concept… but it is poorly named. One reason it is poorly named is it is too vague of an expression… the second reason it is poorly named is it is incomplete. It should say Resource Acquisition is Initialization and De-initialization… because the real issue is cleaning up your resources when you are done using them.

What does it mean by 'Resource'? All it really means is that a particular computer resource such as memory(memory pointer), a file pointer, a network socket connection, a database connection… etc…etc…etc… that needs to be *created and uncreated(*deallocated) should be wrapped in an object. This allows the constructor and destructor of that object to clean itself up.

Smart pointers exist because of the RAII concept… they are chunks of allocated memory that are automatically(automagically) freed up when the smart pointer go out of scope. Even in extraneous conditions, such as an exception, the objects are guaranteed to be destroyed.

Look at the following example:

```
void destruct() {
    vector<int> vi(10);
    auto p = new A[3];
    auto p2 = make_unique<A[]>(3);

    vi.at(10000) = 42;

    delete[] p;
}
```

There are two types of pointers: a raw pointer 'p' and a smart pointer p2. Notice that an **out of bounds exception** will happen at the line at: ***vi.at(10000) = 42;*** This will skip the delete[] p at the end of the function and thus we will have a memory leak.

However, the smart pointer p2 will go out of scope and thus it will have its destructor called regardless of an exception of not. So while RAII is a bad acronym it is a great concept. So let's try an exercise…

15. Let's say you are connecting to a MySQL/Maria database using its C API functions to retrieve some data. We want to encapsulate the connection, which is a .

    The database API's we are concerned with is as follows:

```
MYSQL * mysql_init();
MYSQL * mysql_real_connect(MYSQL * mysql,
                           const char * host,
                           const char * user,
                           const char * passwd);
void mysql_close(MYSQL * mysql);
```

Example usage in a C program:

```
//init MYSQL C struct
MYSQL* mp = mysql_init();
mp = mysql_real_connect(mp,"local","admin","badpassword");
// do some work….
// now close
mysql_close(mp);
```

a) First thing to do is make some **fake** MySQL functions for 'testing'. The That means:
   a. Make an empty struct called MYSQL
   b. Make a function called mysql_init() that allocates a MYSQL struct and returns the pointer. For allocation you can use new or malloc…
   c. Create a mysql_real_connect() function that takes a MYSQL pointer and three char arrays. The function itself does NOTHING (because this is just a test after all…)
   d. Create a mysql_close() function that simply takes a pointer to a MYSQL and frees the memory it points to.

b) Now create a class wrapper called mysql_wrapper that is responsible for creating and destroying the connection to the database. This means:
   a. The constructor takes three string arguments: *host, user* and *password*. Save these in member variables of the class as they will be used with calling *init*() method below.
   b. Create an *init()* method that takes no arguments. This method will call *mysql_init()* and *mysql_real_connect()* passing the host, user and password set in the constructor. The MYSQL pointer that is returned from these functions should be saved as a member of the class (make it public). *init()* returns a bool if it could no allocate the memory.
   c. Create a method called *disconnect()* that calls mysql_close() IF and ONLY IF the init() method had been called and the pointer is not nullptr. Otherwise just return.
   d. Create a destructor that frees up the connection to the database by calling the *disconnect()* method.

c) Test out the code with the sample:

```
void test() {
    mysql_wrapper  msql("localhost", "admin", "badpassword");
    if (!mysql.init()) {
        cout << "oops\n";
        return;
    }
    auto myptr = msql.myptr;  // this would be the internal mysql pointer
    // mysql C api calls would be called here

}   // end of function… the database should be disconnected when it gets to here…
```

Special Note: The MySQL/Maria DB C API functions I gave you are not exactly right. Some of the functions take many more arguments but to keep things simple I reduced it down.

To see the actual C API go to the following page https://mariadb.com/kb/en/mariadb-connectorc-api-functions/

Also there is a C++ wrapper for the API already in existence that performs ALL the database functions here: https://github.com/viaduck/mariadbpp